COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	Sep 8, 2026	**DATE SUBMITTED:**	**Sep 12, 2026**
NAME:	Yazinel Corro	**INSTRUCTOR:**	Engr.Jimlord Quejado

ACTIVITY NO. 7.1 Hands-on Activity 7.1 Sorting Algorithms Pt1

1. PROCEDURE OUTPUT

ILO A

Main

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

//bubble sort

// generate 100 random values
// create a function that will display the random generated values

const int maxSize = 100;

void displayArr(int arr[]);

int main() {
    // create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize];

    // use time(0) as seed for random values
    std::srand(std::time(0));

    // populate the array with random values
    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = std::rand() % 100;
    }

    std::cout << "Unsorted Array:\n";
    displayArr(unsortedArr);

    //std::cout << "\nBubble Sort:\n";
    //bubbleSort(unsortedArr, maxSize);

    //std::cout << "\nInsertion Sort:\n";
    //insertionSort(unsortedArr, maxSize);

    std::cout << "\nSelection Sort:\n";
```

```
27      std::cout << "Unsorted Array:\n";
28      displayArr(unsortedArr);
29
30      //std::cout << "\nBubble Sort:\n";
31      //bubbleSort(unsortedArr, maxSize);
32
33      //std::cout << "\nInsertion Sort:\n";
34      //insertionSort(unsortedArr, maxSize);
35
36      std::cout << "\nSelection Sort:\n";
37      selectionSort(unsortedArr, maxSize);
38      displayArr(unsortedArr);
39
40      return 0;
41  }
42
43  void displayArr(int arr[]) {
44      for (int i = 0; i < maxSize; i++) {
45          std::cout << arr[i] << " ";
46      }
47      std::cout << std::endl;
48  }
49
50
```

Searching Algo

```cpp
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

#include <iostream>

// Bubble Sort
template <typename T>
void bubbleSort(T arr[], int arrSize) {
    for (int i = 0; i < arrSize - 1; i++) {
        for (int j = 0; j < arrSize - i - 1; j++) {
            // Compare adjacent elements and swap if out of order
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
            }
        }
    }
}

//insertion sort
template <typename T>
void insertionSort(T arr[], int arrSize) {
    int k = 1, j;
    T temp;

    while (k < arrSize) {
        temp = arr[k];
        j = k - 1;

        // Added 'j >= 0' to prevent out-of-bounds array access (arr[-1])
        while (j >= 0 && temp < arr[j]) {
            arr[j + 1] = arr[j];
            j--;   // responsible for swapping
        }
        arr[j + 1] = temp; // responsible for swapping
        k++; // responsible for swapping
    }
}
```

```cpp
//helper function

template <typename T>
int routine_smallest(T A[], int maxSize, int k){ //int k = current index;

    int position, j;
    T smallestElement = A[k];

    for (int J = k+1; J < maxSize; J++) {
        if (A[J] < smallestElement) {
            smallestElement = A[J];
            position = J;
        }
    }

    return position;
}

//selection sort
template <typename T>
void selectionSort(T arr[], int arrSize) {
    int POS, temp;

    for (int i = 0; i < arrSize; i++) {
        POS = routine_smallest(arr, arrSize, i);

        temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;

    }

```

```cpp
//selection sort
template <typename T>
void selectionSort(T arr[], int arrSize) {
    int POS, temp;

    for (int i = 0; i < arrSize; i++) {
        POS = routine_smallest(arr, arrSize, i);

        temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;

    }

}

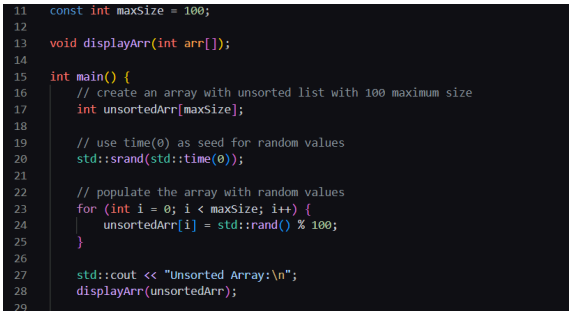

#endif

```

OUTPUT

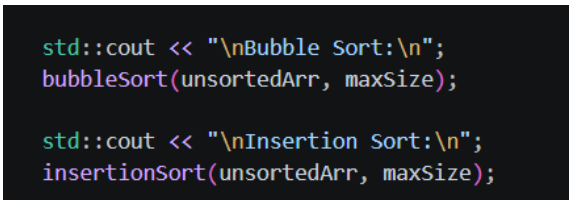
```
Unsorted Array:
37 66 53 77 34 52 52 87 19 61 27 71 50 12 55 5 41 3 59 7 28 61 35 4 88 6 67 78 35 8 24 83 49 37 31 22 3 31 56 85 4 95
30 5 74 85 64 96 34 24 43 98 34 69 97 3 42 80 5 56 89 39 44 52 53 33 36 81 53 66 70 67 26 98 58 27 88 99 77 56 11 22 1
0 31 81 52 18 69 90 81 64 26 37 98 21 22 57 88 58 38

Selection Sort:
3 3 3 4 4 5 5 5 6 7 8 10 11 50 12 18 19 21 22 22 22 24 24 26 26 27 27 28 30 31 31 31 33 34 34 34 35 35 36 37 37 37 38
39 41 42 43 44 49 52 52 52 52 53 53 53 55 56 56 56 57 58 58 59 61 61 64 64 66 66 67 67 69 69 70 71 74 77 77 78 80 81 8
1 81 83 85 85 87 88 88 88 89 90 95 96 97 98 98 98 99
PS C:\Users\yazinel\OneDrive\Desktop\cpe 010 - corro-hoa1\HOA7P1>
```

Code + Console Screenshot	
Observations	My observation is that the program successfully creates a 100-element array of random numbers. The values are initially unsorted and they are given as input to the different sorting algorithm

```
11  const int maxSize = 100;
12
13  void displayArr(int arr[]);
14
15  int main() {
16      // create an array with unsorted list with 100 maximum size
17      int unsortedArr[maxSize];
18
19      // use time(0) as seed for random values
20      std::srand(std::time(0));
21
22      // populate the array with random values
23      for (int i = 0; i < maxSize; i++) {
24          unsortedArr[i] = std::rand() % 100;
25      }
26
27      std::cout << "Unsorted Array:\n";
28      displayArr(unsortedArr);
```

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	
Observations	The Bubble Sort algorithm compares adjacent elements and swaps them when they are in the wrong order. As the process repeats, larger values move toward the end until the array is sorted in ascending order.

```
std::cout << "\nBubble Sort:\n";
bubbleSort(unsortedArr, maxSize);

std::cout << "\nInsertion Sort:\n";
insertionSort(unsortedArr, maxSize);
```

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	
Observations	My Observation is the Selection Sort searches for the smallest element in the unsorted part of the array and puts it in the right place. This process continues until the entire array is sorted in ascending order.

Table 7-3. Selection Sort Algorithm

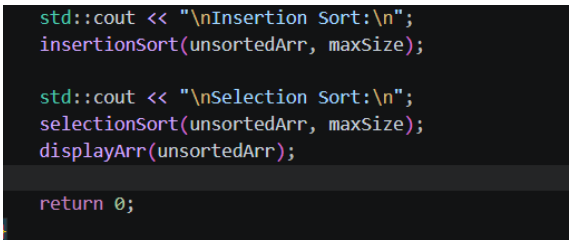
Code + Console Screenshot	
Observations	My observation is that in the insertion sort, the ordered array is built one element at a time. It moves larger elements to the right and inserts the current element in the correct place.

Table 7-4. Insertion Sort Algorithm

SUPPLEMENTARY ACTIVITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

List of Candidates

Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate

CODE

Main

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <string>
#include "selectionAlgo.h"

const int maxSize = 101;

void displayArr(int arr[]);

int main()
{
    std::srand(std::time(0));

    int unsortedArr[maxSize];

    // Generate random votes from 1 to 5
    for (int i = 0; i < maxSize; i++)
    {
        unsortedArr[i] = (std::rand() % 5) + 1;
    }

    std::string candidates[5] = {
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Banaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    std::cout << "Unsorted Array:\n";
    displayArr(unsortedArr);

    // Sort the votes
    selectionSort(unsortedArr, maxSize);

```

```
11  int main()
23      std::string candidates[5] = {
25          "Cornelius Raymon Agustin",
26          "Deja Jayla Banaga",
27          "Lalla Brielle Yabut",
28          "Franklin Relano Castro"
29      };
30
31      std::cout << "Unsorted Array:\n";
32      displayArr(unsortedArr);
33
34      // Sort the votes
35      selectionSort(unsortedArr, maxSize);
36
37      std::cout << "\nSorted Array:\n";
38      displayArr(unsortedArr);
39
40      // Manual count
41      int manualCount[6] = {0};
42
43      for (int i = 0; i < maxSize; i++)
44      {
45          manualCount[unsortedArr[i]]++;
46      }
47
48      // Algorithm count
49      int voteCount[6] = {0};
50
51      for (int i = 0; i < maxSize; i++)
52      {
53          int vote = unsortedArr[i];
54
55          if (vote >= 1 && vote <= 5)
56          {
57              voteCount[vote]++;
58          }
```

```
61      // Find the winner
62      int winner = 1;
63
64      for (int i = 2; i <= 5; i++)
65      {
66          if (voteCount[i] > voteCount[winner])
67          {
68              winner = i;
69          }
70      }
71
```

```
    // Display results
    std::cout << "\nVote Count Results:\n";

    for (int i = 1; i <= 5; i++)
    {
        std::cout << "\nCandidate " << i << ": ";
        std::cout << candidates[i - 1] << "\n";

        std::cout << "Manual Count: ";
        std::cout << manualCount[i] << "\n";

        std::cout << "Algorithm Count: ";
        std::cout << voteCount[i] << "\n";
    }

    std::cout << "\nWinner: Candidate " << winner << "\n";
    std::cout << "Name: "
              << candidates[winner - 1] << "\n";
    std::cout << "Votes: " << voteCount[winner] << "\n";

    return 0;
}

void displayArr(int arr[])
{
    for (int i = 0; i < maxSize; i++)
    {
        std::cout << arr[i] << " ";

        if ((i + 1) % 20 == 0)
        {
            std::cout << "\n";
        }
    }
```

```
void displayArr(int arr[])
{
    for (int i = 0; i < maxSize; i++)
    {
        std::cout << arr[i] << " ";

        if ((i + 1) % 20 == 0)
        {
            std::cout << "\n";
        }
    }

    std::cout << "\n";
}
```

Selection Algo

```cpp
#ifndef SELECTIONALGO_H
#define SELECTIONALGO_H

// Find the smallest element
template <typename T>
int routine_smallest(T A[], int maxSize, int k)
{
    int position = k;
    T smallestElement = A[k];

    for (int j = k + 1; j < maxSize; j++)
    {
        if (A[j] < smallestElement)
        {
            smallestElement = A[j];
            position = j;
        }
    }

    return position;
}
// Selection Sort
template <typename T>
void selectionSort(T arr[], int arrSize)
{
    for (int i = 0; i < arrSize - 1; i++)
    {
        int pos = routine_smallest(
            arr, arrSize, i
        );

        T temp = arr[i];
        arr[i] = arr[pos];
        arr[pos] = temp;
    }
}
#endif
```

OUTPUT OF THE CODE

```
Unsorted Array:
4 3 1 2 4 3 3 1 2 3 1 1 1 2 3 4 1 1 1 1
1 1 4 4 5 5 5 1 1 4 1 3 1 5 4 4 4 2 4 3
5 3 1 3 5 3 3 5 5 3 4 2 4 3 4 5 5 5 4 2
5 2 3 2 4 4 1 2 1 1 4 5 5 5 2 4 5 3 4 2
1 1 1 1 3 4 4 1 3 4 4 1 1 4 2 2 1 1 1 2
5

Sorted Array:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2
2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
5
```

```
Vote Count Results:

Candidate 1: Bo Dalton Capistrano
Manual Count: 29
Algorithm Count: 29

Candidate 2: Cornelius Raymon Agustin
Manual Count: 14
Algorithm Count: 14

Candidate 3: Deja Jayla Banaga
Manual Count: 17
Algorithm Count: 17

Candidate 4: Lalla Brielle Yabut
Manual Count: 24
Algorithm Count: 24

Candidate 5: Franklin Relano Castro
Manual Count: 17
Algorithm Count: 17

Winner: Candidate 1
Name: Bo Dalton Capistrano
Votes: 29
```

Manual Count	Count Result of Algorithm
Candidate 1 = **17**	Candidate 1 = **17**
Candidate 2 = **29**	Candidate 2 = **29**
Candidate 3 = **18**	Candidate 3 = **18**
Candidate 4 = **17**	Candidate 4 = **17**
Candidate 5 = **20**	Candidate 5 = **20**

Question: Was your developed vote counting algorithm effective? Why or why not?

- Yes, the developed vote counting algorithm was found to be good as the result of the algorithm was exactly the same as the counts that were done manually for all the candidates, which means the algorithm was able to verify the votes within the range of 1 to 5 correctly, increment the candidate tally and no errors were found.

CONCLUSIONS

Summary of lessons learned:

- I learned how basic sorting algorithms like Bubble Sort, Insertion Sort, and Selection Sort work in C++. I also learned how sorting array elements makes it much easier to organize, search, and tally duplicate data.

Analysis of the procedure:

- The main procedure demonstrated how three sorting algorithms handle 100 random numbers. Bubble Sort repeatedly swaps adjacent items, Insertion Sort shifts elements to insert values in order, and Selection Sort finds the smallest value to place at the front

Analysis of the supplementary activity:

- The supplementary activity applied Selection Sort to solve an election vote-counting problem. Sorting the array grouped identical candidate votes together, which made vote tallying simple and verified that Candidate 1 won with 29 votes.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

- I performed well in this activity by successfully writing the code, using header files, and matching the algorithm results with manual counting. My main area for improvement is practicing faster sorting methods that handle much larger sets of data.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: code assistance

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.